

Pipeline 2 — MongoDB CRUD App

CI/CD GitLab · Docker · Kubernetes · AWS · Terraform

Hasna Daoui — Module DevSecOps 2025/2026

1. Présentation du pipeline

Ce pipeline CI/CD déploie une application Node.js MongoDB CRUD. Il est composé de 3 stages : build de l'image Docker, push dans le GitLab Container Registry, et déploiement sur Kubernetes. L'infrastructure AWS EC2 est provisionnée avec Terraform.

Outils utilisés : GitLab CI/CD, Docker, Kubernetes, Terraform, AWS EC2, MongoDB, Node.js/Express.

2. Stages CI/CD

Stage 1 — build:docker

Build de l'image Docker depuis le Dockerfile de l'application, puis push dans le GitLab Registry avec deux tags : le SHA du commit et latest.

Stage 2 — push:registry

Re-tag de l'image avec le nom de la branche (main) et re-push dans le registry. Dépend du job build:docker via la directive needs.

Stage 3 — deploy:kubernetes

Déploiement sur Kubernetes via un runner local (shell/Windows). Configure kubectl depuis la variable KUBE_CONFIG_BASE64, applique deploy.yaml et vérifie le rollout.

#2388906546

✓ Passed Created 2 months ago by **Hasna Daoui**, finished 2 months ago
For commit `b9886bd9` [Edit .gitlab-ci.yml](#)
n `main`
`latest` `branch` 3 jobs 1.16 1 minute 44 seconds, queued for 1 seconds
Pipeline Jobs 3 Tests 0

Group jobs by Stage Job dependencies

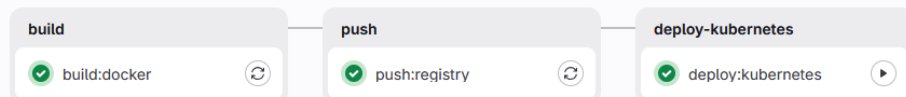


Figure 1 — Pipeline passé : 3 stages réussis (build, push, deploy)

3. Infrastructure AWS avec Terraform

L'instance EC2 (Amazon Linux 2, t2.micro) est provisionnée avec Terraform. La commande `terraform apply` retourne l'IP publique : 34.207.143.116.

```
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no
differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

public_ip = "34.207.143.116"
P5 C:\Users\user\Desktop\MongodbApp+Kubernities+AWS+Gitlab\teraform aws>
```

Figure 3 — Terraform apply : infrastructure à jour, IP publique 34.207.143.116

4. Connexion SSH à l'EC2

```
C:\Users\user\Desktop\MongodApp>kubernities+AWS+Gitlab terraform aws>ssh -i terraform-key.pem ec2-user@304.207.143.116
The authenticity of host '304.207.143.116 (304.207.143.116)' can't be established.
ED25519 key fingerprint is SHA256:3aus+WwNciBhtaNOL8B+F0ftc/gsmWeSrXVrdv3Apus.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '304.207.143.116' (ED25519) to the list of known hosts.
```



```
#_
#####      Amazon Linux 2
#####
#####}####
##### \###
#####  \#/_
#####   V=^ ^-->
#####    ^ _
#####   _/_/
#####  _/_/
##### _/m/'      AL2 End of Life is 2026-06-30.

                        A newer version of Amazon Linux is available!

                        Amazon Linux 2023, GA and supported until 2028-03-15.
                        https://aws.amazon.com/linux/amazon-linux-2023/
```

```
[ec2-user@ip-172-31-17-99 ~]$ docker status
docker: 'status' is not a docker command.
See 'docker --help'

[ec2-user@ip-172-31-17-99 ~]$ docker --version
Docker version 25.0.14, build 0bab007

[ec2-user@ip-172-31-17-99 ~]$ docker login registry.gitlab.com
Username: cyber2026
Password:
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
```

Figure 4 — SSH sur l'EC2, Docker version 25.0.14 confirmé

```
C:\Users\user\Desktop\MongodbApp+Kubernities+AWS+GitLab\terraform aws>ssh -i terraform-key.pem -L 3000:localhost:3000 ec2-user@34.207.143.116
Last login: Sun Mar 15 15:03:25 2026 from 196.200.150.66

#_
##### Amazon Linux 2
#####
#####| AL2 End of Life is 2026-06-30.
#####|
#####|_#_
#####V~' i-->

A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-172-31-17-99 ~]$
```

Figure 5 — SSH avec port forwarding -L 3000:localhost:3000

5. Déploiement Docker sur EC2

Création du réseau Docker appnet, démarrage du conteneur MongoDB puis du conteneur de l'application avec la variable d'environnement MONGO_URI.

```
Login Succeeded
[ec2-user@ip-172-31-17-99 ~]$ docker network create appnet
489ab176d4289c6dc3ad303adf00c2677c9930758964ef054c66708ce7b340e0
```

Figure 6 — Création du réseau Docker 'appnet'

```
[ec2-user@ip-172-31-17-99 ~]$ docker run -d \
> --name mongo \
> --network appnet \
> -p 27017:27017 \
> mongo:7
Unable to find image 'mongo:7' locally
7: Pulling from library/mongo
b1cba2e842ca: Pull complete
7c6daea0607a: Pull complete
488b7536aa90: Pull complete
053e1e2811d6: Pull complete
f80d1f18be1a: Pull complete
70624009c824: Pull complete
c1575978db83: Pull complete
e7873cabff83: Pull complete
Digest: sha256:adfc10cca964f99224ac4948e782c260c4c87e71958df334e8ca882ff85062ea
Status: Downloaded newer image for mongo:7
cb125741a5f94f2c8b22e4f12b4f84277079b70b4958c0be881e3f97a4d63ab1
```

Figure 7 — Démarrage du conteneur MongoDB (mongo:7) sur appnet

```
[ec2-user@ip-172-31-17-99 ~]$ docker run -d --name mongo-crud-app --network appnet -p 3000:3000 -e MONGO_URI="mongodb://mongo:27017" registry.gitlab.com/cyber2026/mongodbusersapp/mongo-crud-app:latest
bc3c9991672008337aaed2d9651084bb5bfc43143fad3231c8f8dc6520b3237
[ec2-user@ip-172-31-17-99 ~]$ docker stop mongo-crud-app
mongo-crud-app
*[[[ec2-user@ip-172-31-17-99 docker rm mongo-crud-app
mongo-crud-app
[ec2-user@ip-172-31-17-99 ~]$ docker run -d \
> --name mongo-crud-app \
> --network appnet \
> -p 3000:3000 \
> -e MONGO_URI="mongodb://mongo:27017" \
> registry.gitlab.com/cyber2026/mongodbusersapp/mongo-crud-app:latest
5e845d2cbbf725d5b0fe0bd6f5e25e2ccaceffee37cd59d986f7ea2b9c767eaf
[ec2-user@ip-172-31-17-99 ~]$
```

Figure 8 — Démarrage du conteneur app depuis le GitLab Registry

6. Test Docker local (docker-compose)

Avant le déploiement cloud, l'application est testée localement via docker-compose up --build.

```
PS C:\Users\user\Desktop\MongodbApp+Kubernities+AWS+Gitlab\mongo-crud-app> docker-compose up
--build
[+] up 2/2
  ✓ Image mongo-crud-app-app Built 6.3s
  ✓ Container mongo-crud-app Recreated 0.6s
Attaching to mongo-crud-app, mongodb
mongo-crud-app | > mongo-crud-app@1.0.0 start
mongo-crud-app | > node server.js
mongodb | {"t":{"$date":"2026-03-12T22:04:33.571+00:00"},"s":"I", "c":"-",
"id":8991200, "ctx":"main", "msg":"Shuffling initializers", "attr":{"seed":2391678928}}
mongodb | {"t":{"$date":"2026-03-12T22:04:33.657+00:00"},"s":"I", "c":"CONTROL",
"id":97374, "ctx":"main", "msg":"Automatically disabling TLS 1.0 and TLS 1.1, to force-enab
le TLS 1.1 specify --sslDisabledProtocols 'TLS1_0'; to force-enable TLS 1.0 specify --sslD
isabledProtocols 'none'"}
mongodb | {"t":{"$date":"2026-03-12T22:04:33.704+00:00"},"s":"I", "c":"NETWORK"
```

Figure 9 — docker-compose : app démarrée, MongoDB initialisé

7. Déploiement Kubernetes

Le fichier deploy.yaml définit un Namespace (mongodb-app), un PVC MongoDB, un Deployment MongoDB (1 replica), un Deployment app (3 replicas), et deux Services (ClusterIP pour MongoDB, LoadBalancer pour l'app sur le port 80).

```
36 14:34:45 eplicas are available...
37 14:34:45 Waiting for deployment "mongo-crud-app" rollout to finish: 2 of 3 updated r
eplicas are available...
38 14:34:46 deployment "mongo-crud-app" successfully rolled out
39 14:34:46 $ kubectl rollout status deployment/mongo -n default
40 14:34:47 deployment "mongo" successfully rolled out
41 14:34:47 $ kubectl get pods
42 14:34:47 NAME READY STATUS RESTARTS AGE
43 14:34:47 mongo-79c8f977bb-cpqz9 1/1 Running 0 37s
44 14:34:47 mongo-crud-app-858f977866-hxgj4 1/1 Running 0 37s
45 14:34:47 mongo-crud-app-858f977866-mh5qz 1/1 Running 0 37s
46 14:34:47 mongo-crud-app-858f977866-v79dx 1/1 Running 0 37s
47 14:34:47 $ kubectl get svc
48 14:34:47 NAME TYPE CLUSTER-IP EXTERNAL-IP PORT
(S) AGE
49 14:34:47 kubernetes ClusterIP 10.96.0.1 <none> 443/T
CP 13d
50 14:34:47 mongo-crud-app-service LoadBalancer 10.96.77.158 <pending> 80:32
098/TCP 37s
51 14:34:47 mongo-service ClusterIP 10.97.155.144 <none> 2701
```

Figure 10 — kubectl get pods : 4 pods Running (1 mongo + 3 app)

```
PS C:\Users\user> kubectl logs -l app=mongo-crud-app --tail=5
> mongo-crud-app@1.0.0 start
> node server.js

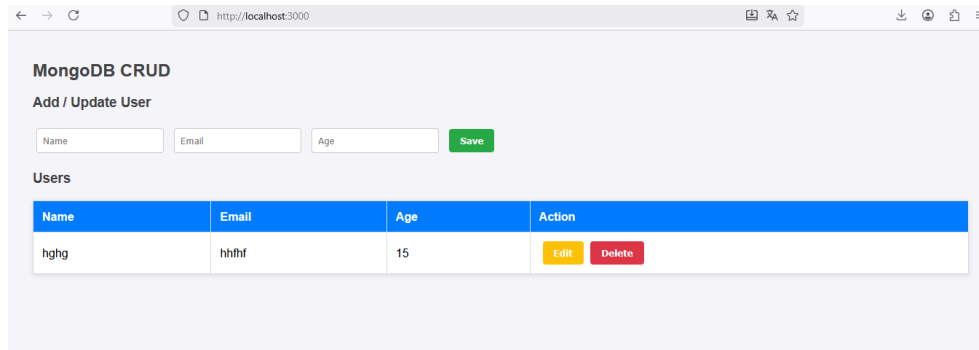
Server running: http://localhost:3000
MongoDB connected
> mongo-crud-app@1.0.0 start
> node server.js

Server running: http://localhost:3000
MongoDB connected
```

Figure 11 — kubectI logs : Server running :3000 et MongoDB connected

8. Test de l'application

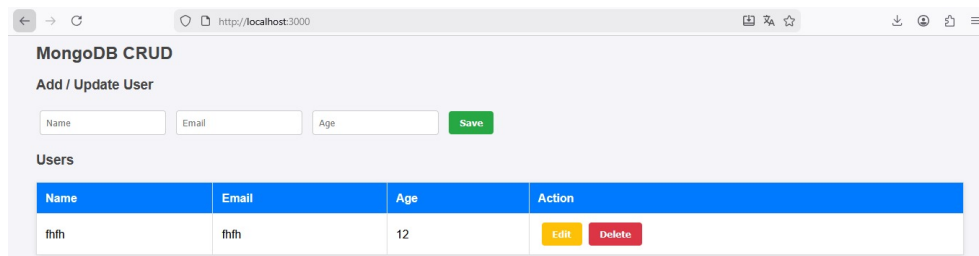
L'application MongoDB CRUD est accessible sur <http://localhost:3000>. Elle permet d'ajouter, modifier et supprimer des utilisateurs (Nom, Email, Age).



The screenshot shows a web browser at <http://localhost:3000> displaying the 'MongoDB CRUD' application. The 'Add / Update User' section has three input fields: 'Name' (containing 'hghg'), 'Email' (containing 'hhfhf'), and 'Age' (containing '15'), followed by a green 'Save' button. Below this, the 'Users' section contains a table with one user entry.

Name	Email	Age	Action
hghg	hhfhf	15	<button>Edit</button> <button>Delete</button>

Figure 12 — App CRUD : utilisateur 'hghg', email hhfhf, age 15



The screenshot shows the same MongoDB CRUD application interface. The 'Add / Update User' section has three input fields: 'Name' (containing 'fhfh'), 'Email' (containing 'fhfh'), and 'Age' (containing '12'), followed by a green 'Save' button. Below this, the 'Users' section contains a table with one user entry.

Name	Email	Age	Action
fhfh	fhfh	12	<button>Edit</button> <button>Delete</button>

Figure 13 — App CRUD : utilisateur 'fhfh', age 12

9. Conclusion

Le pipeline est entièrement fonctionnel. Les 3 stages GitLab CI/CD passent avec succès, les 4 pods Kubernetes sont Running, MongoDB est connecté, et l'application CRUD est accessible. L'infrastructure AWS est provisionnée automatiquement via Terraform.